Regularização

Resumo da Aula 7.1

Objetivo: evitar que um modelo se ajuste demais aos dados de treino e tenha dificuldade com dados novos. A solução estudada é a regularização, aplicada à regressão linear por meio de Ridge, Lasso e ElasticNet.

A pergunta da aula: como fazer o modelo aprender um padrão que também funcione em exemplos novos?

1. Onde isso entra no machine learning?

Estamos em **aprendizado supervisionado para regressão**: usamos exemplos com características e respostas conhecidas para aprender a prever um número.

Imagine prever o preço de uma casa usando área, quartos e idade do imóvel. O modelo aprende uma fórmula que combina essas informações. Os pesos dessa fórmula são os **coeficientes**.

Num exemplo simplificado, se o modelo aprender "preço = área $\times$ 3 mil", ele prevê R$ 240 mil para uma casa de 80 m^2. Aprender, nesse caso, é encontrar os números da fórmula.

2. Aprender bem não é ajustar tudo ao treino

- **Underfitting:** o modelo não representa bem o padrão. O erro costuma ser alto no treino e na validação.
- **Bom ajuste:** aprende uma relação que também funciona em exemplos novos.
- **Overfitting:** ajusta-se demais às particularidades do treino. O erro fica baixo no treino, mas maior na validação.

É como decorar respostas de exercícios e ter dificuldade quando a pergunta muda. No ML, queremos **generalização**: bom desempenho em dados que não participaram do treinamento.

3. Como a regularização ajuda?

A regressão comum procura reduzir o erro das previsões. A regularização acrescenta uma penalidade sobre os coeficientes, equilibrando o ajuste aos dados e o tamanho dos pesos.

$$\text{Objetivo} = \text{erro de treino} + \alpha \times \text{penalidade}.$$

O erro de treino pode subir um pouco e o de validação melhorar. Isso é uma possibilidade que verificamos nos dados, não uma garantia.

4. O que muda entre Ridge, Lasso e ElasticNet?

- **Ridge (L2):** usa a soma dos quadrados dos coeficientes como penalidade. Encolhe os pesos; em geral, não os zera exatamente.
- **Lasso (L1):** usa a soma dos valores absolutos dos coeficientes. Pode zerar pesos e retirar a contribuição de características.
- **ElasticNet:** mistura as penalidades L1 e L2. Combina encolhimento e possibilidade de zerar pesos.

Exemplo: se o Lasso zerar o coeficiente da idade do imóvel, essa característica deixa de contribuir para a previsão daquele modelo.

Isso não prova que a idade seja sempre irrelevante. A escolha depende dos exemplos, da intensidade da penalidade e das relações entre as características.

O **intercepto**, termo constante da fórmula, normalmente não recebe a penalidade apresentada na aula.

5. Alpha controla a intensidade

- **Alpha pequeno:** penalidade fraca; permite coeficientes maiores.
- **Alpha intermediário:** pode equilibrar ajuste e generalização.
- **Alpha muito grande:** penalidade forte; pode prejudicar o ajuste e causar underfitting.

Alpha é um hiperparâmetro: nós escolhemos seu valor durante a comparação. Os coeficientes são parâmetros aprendidos pelo modelo durante o treinamento.

O melhor alpha é escolhido pela **validação**, não pelo menor erro de treino. Ridge e Lasso usam escalas diferentes nas funções objetivo; o mesmo número de alpha não significa a mesma intensidade nos dois.

6. A mistura do ElasticNet

O `l1_ratio` controla a mistura: próximo de 1 dá mais participação à L1; próximo de 0, à L2. Ele controla a composição da penalidade, enquanto alpha controla sua força total.

7. Por que padronizar os dados?

Características podem ter unidades e escalas diferentes. Como a penalidade depende do tamanho dos coeficientes, mudar a unidade de uma característica pode alterar a regularização.

O **StandardScaler** subtrai a média e divide pelo desvio padrão de cada característica. Isso coloca as entradas em escalas comparáveis.

Essas estatísticas devem ser aprendidas **somente no treino**. Durante a validação cruzada, isso se repete dentro de cada rodada. O **Pipeline** encadeia a padronização e o modelo para organizar esse processo e evitar vazamento de dados.

8. Como comparar os valores de alpha?

Na **validação cruzada com cinco dobras**, dividimos o conjunto de treino em cinco partes. Em cada rodada, quatro treinam e uma valida. Repetimos até cada parte ter servido como validação.

Para cada configuração, calculamos a média dos erros das cinco rodadas. O desvio padrão mostra quanto os erros variaram entre elas. A comparação deve usar as mesmas divisões para os modelos.

9. O que o RMSE nos diz?

O **RMSE** mede o erro das previsões na mesma unidade do alvo: **quanto menor, melhor**. Ele é a raiz da média dos erros ao quadrado. RMSE-CV é a média dos RMSEs obtidos na validação cruzada.

Por exemplo, se o alvo está em reais, o RMSE também está em reais. Ele não representa uma porcentagem de acertos.

10. Cada conjunto tem seu papel

- **Treino:** aprender os coeficientes da fórmula.
- **Validação:** comparar configurações e escolher modelo e hiperparâmetros. Pode ser feita por dobras dentro do treino.
- **Teste:** avaliar a escolha final em exemplos reservados. Não usar esse resultado para reajustar o modelo.

Fonte: Aula 7.1 - Regularização, especialmente slides 5 a 41. Exemplos simplificados para estudo.